

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mohd. Razia Alangir Banu
Student Name	Y. Yashwanth
Roll Number	2520030506
Branch	CSE
Course Outcome	CO-5

Case Study Topic: Strategic Inventory Investment Minimization: Balancing Risk and Budget via the Unbounded Knapsack Problem

Work Done Summary:

A nationwide logistics and retail distributor manages a central mega-warehouse. The supply chain director is allocated a strict maximum procurement weight capacity of W kilograms for a high-priority restocking window. There are N different categories of products available from vendors. Each item type i has a fixed weight w_i and yields a distinct profit margin v_i upon eventual retail distribution. Due to wholesale bulk agreements, the vendor has an infinite supply of each product category, meaning the distributor can select any product type multiple times (unbounded supply).

The enterprise faces multiple constraints and strategic challenges:

1. **Capacity Overruns:** Selecting items blindly without considering weight efficiency causes inventory capacity overflows, triggering heavy warehouse overflow fines.
2. **Sub-optimal Multipliers:** Simple greedy selection models (e.g., picking items solely based on the highest profit or highest density) fail to find the globally optimal inventory combination when item weights do not divide evenly into the remaining space.
3. **Combinatorial Explosion:** The inventory manager needs a system that scales cleanly to large target capacities without stalling processing pipelines.

To maximize inventory value, the software engineering group must transition from manual heuristics to an optimized Dynamic Programming (DP) framework. They will model this problem as the Unbounded Knapsack Problem to find the absolute maximum value achievable under strict weight constraints.

Implementation Details:

1. Failure of Greedy Selection in Discrete Knapsack Problems

The greedy approach for knapsack problems calculates the profit density ($\frac{\text{profit}}{\text{weight}}$) for each item and selects items with the highest density first.

In the **Fractional Knapsack** problem, items can be broken down into smaller pieces. The greedy strategy works perfectly here because any remaining capacity can be filled with a precise fraction of the next best item.

In the **Discrete (Unbounded) Knapsack** problem, items must be taken as whole units. If we use the greedy approach, we might be left with small, empty gaps of unused capacity that cannot fit another heavy, high-value item. This unused space goes to waste. A dynamic programming approach checks all possible item combinations to find a mix that maximizes total value, even if it means picking individual items with lower initial densities to minimize wasted capacity.

2. Dynamic Programming Properties & Mathematical Recurrence

The problem exhibits two core properties required for Dynamic Programming:

- **Optimal Substructure:** The optimal choice for a total capacity w depends on the optimal choices made for smaller capacities ($w - w_i$).
- **Overlapping Subproblems:** During evaluation, the maximum profit for specific smaller weight capacities is calculated repeatedly across different item combination branches. Storing these values prevents redundant calculations.

The bottom-up configuration uses a 1D array dp of size $W + 1$, where $dp[w]$ stores the maximum profit achievable with a capacity of w . The mathematical recurrence relation is:

$text\{dp\}[w] = \max_{0 \leq i < N} (text\{dp\}[w], text\{profit\}[i] + text\{dp\}[w - text\{weight\}[i]])$ quad $text\{where\} \ w \geq text\{weight\}[i]$
 Base Case: $text\{dp\}[0] = 0$ (Zero capacity yields zero profit).

3. Step-by-Step Dynamic Programming Trace

Given: $W = 8$, items: $I_1(2, 30)$, $I_2(3, 50)$, $I_3(4, 60)$

Initialize array dp of size 9 with all 0s: $[0, 0, 0, 0, 0, 0, 0, 0, 0]$

- **$w = 0, 1$:** No items can fit because the minimum item weight is 2. $dp[0]=0$, $dp[1]=0$.
- **$w = 2$:** Only $I_1(2, 30)$ fits. $text\{dp\}[2] = \max(0, 30 + text\{dp\}[0]) = 30$.
- **$w = 3$:** I_1 fits: $\max(0, 30 + text\{dp\}[1]) = 30$. $I_2(3, 50)$ fits: $\max(30, 50 + text\{dp\}[0]) = 50$. $dp[3] = 50$.
- **$w = 4$:** * Try I_1 : $\max(0, 30 + text\{dp\}[2]) = 30 + 30 = 60$.
 ○ Try I_2 : $\max(60, 50 + text\{dp\}[1]) = 50 + 0 = 60$.
 ○ Try I_3 : $\max(60, 60 + text\{dp\}[0]) = 60 + 0 = 60$. $dp[4] = 60$.
- **$w = 5$:**
 ○ Try I_1 : $\max(0, 30 + text\{dp\}[3]) = 30 + 50 = 80$.
 ○ Try I_2 : $\max(80, 50 + text\{dp\}[2]) = 50 + 30 = 80$.
 ○ Try I_3 : $\max(80, 60 + text\{dp\}[1]) = 60 + 0 = 60$. $dp[5] = 80$.
- **$w = 6$:**
 ○ Try I_1 : $\max(0, 30 + text\{dp\}[4]) = 30 + 60 = 90$.
 ○ Try I_2 : $\max(90, 50 + text\{dp\}[3]) = 50 + 50 = 100$.
 ○ Try I_3 : $\max(100, 60 + text\{dp\}[2]) = 60 + 30 = 90$. $dp[6] = 100$.
- **$w = 7$:**
 ○ Try I_1 : $\max(0, 30 + text\{dp\}[5]) = 30 + 80 = 110$.
 ○ Try I_2 : $\max(110, 50 + text\{dp\}[4]) = 50 + 60 = 110$.
 ○ Try I_3 : $\max(110, 60 + text\{dp\}[3]) = 60 + 50 = 110$. $dp[7] = 110$.
- **$w = 8$:**
 ○ Try I_1 : $\max(0, 30 + text\{dp\}[6]) = 30 + 100 = 130$.
 ○ Try I_2 : $\max(130, 50 + text\{dp\}[5]) = 50 + 80 = 130$.
 ○ Try I_3 : $\max(130, 60 + text\{dp\}[4]) = 60 + 60 = 120$. $dp[8] = 130$.

The final array state is $[0, 0, 30, 50, 60, 80, 100, 110, 130]$. The maximum profit at capacity 8 is **130** (achieved by selecting two units of Product 1 and textually nesting two units of Product 2, or combinations like $2+3+3$).

Result: Screenshots / Output

Java

```
import java.util.Arrays;
```

```
public class InventoryOptimizer {
```

```
    /**
```

```
     * Solves the Unbounded Knapsack problem to maximize inventory profit.
```

```
     * @param weights Array containing the weights of each product category.
```

```
     * @param profits Array containing the profits of each product category.
```

```
     * @param capacity The maximum weight capacity allowed in the warehouse.
```

```
     * @return The absolute maximum profit achievable.
```

```
    */
```

```
    public static int maximizeInventoryValue(int[] weights, int[] profits, int capacity) {
        int n = weights.length;
```

```
        // dp[i] stores the max profit achievable for a sub-capacity of weight i
```

```
        int[] dp = new int[capacity + 1];
```

```
        // Base case initialization
```

```

Arrays.fill(dp, 0);

// Iterate through all sub-capacities from 1 up to the target capacity
for (int w = 1; w <= capacity; w++) {
    for (int i = 0; i < n; i++) {
        if (weights[i] <= w) {
            int alternativeProfit = profits[i] + dp[w - weights[i]];
            if (alternativeProfit > dp[w]) {
                dp[w] = alternativeProfit;
            }
        }
    }
}

return dp[capacity];
}

public static void main(String[] args) {
    // Data points defined in the case study problem scenario
    int[] itemWeights = {2, 3, 4};
    int[] itemProfits = {30, 50, 60};
    int maxWarehouseCapacity = 8;

    System.out.println("Launching Supply Chain Optimization Routing System...");
    System.out.println("Available Item Categories: " + itemWeights.length);
    System.out.println("Target Warehouse Capacity Constraint: " + maxWarehouseCapacity + " kg");

    int optimalProfit = maximizeInventoryValue(itemWeights, itemProfits, maxWarehouseCapacity);

    System.out.println("-----");
    System.out.println("Maximum Achievable Inventory Profit: " + optimalProfit);
    System.out.println("-----");
}
}

```

Plaintext

Launching Supply Chain Optimization Routing System...

Available Item Categories: 3

Target Warehouse Capacity Constraint: 8 kg

Maximum Achievable Inventory Profit: 130

FS

GitHub Link: <https://github.com/Yashwanthwhy/DSA-2/blob/main/Casestudy-5.pdf>

Student Signature	Faculty Signature
-------------------	-------------------

